

Specification-Driven Development

Specification-driven development (SDD) is a software development approach in which a clear written specification is created and agreed on before implementation begins.[cite:1][cite:3] The specification defines expected behavior, interfaces, constraints, and acceptance criteria, while code is treated as the implementation of that agreed intent rather than the place where requirements are discovered.[cite:1][cite:4]

Core idea

At the center of SDD is the idea that the specification is the source of truth for the project.[cite:1][cite:4][cite:15] Instead of relying on scattered chat history, informal assumptions, or evolving verbal instructions, teams maintain a durable artifact that product, engineering, and QA can all review against the same standard.[cite:4][cite:12][cite:15]

In AI-assisted development, this matters even more because the LLM performs better when it is given structured boundaries and explicit success criteria.[cite:4][cite:13][cite:15] A well-written spec narrows ambiguity, reduces off-target generations, and makes it easier to verify whether the generated code is correct.[cite:4][cite:13]

What a spec usually contains

A practical specification often includes the following elements:[cite:1][cite:3][cite:12]

- Problem statement and scope.[cite:1][cite:12]
- Functional requirements and expected behaviors.[cite:1][cite:3]
- Interfaces, inputs, outputs, and data contracts.[cite:3][cite:4]
- Constraints such as performance, security, reliability, or compliance requirements.[cite:3][cite:12]
- Edge cases, failure modes, and assumptions.[cite:4][cite:12]
- Acceptance criteria that define what counts as done.[cite:1][cite:12]

The exact format can vary, but the goal stays the same: make intent explicit enough that implementation and validation can proceed with minimal guesswork.[cite:1][cite:4]

Why teams use it

Teams adopt SDD because it improves alignment before substantial coding begins.[cite:4][cite:12] By forcing clarification early, it helps catch misunderstandings when changes are still cheap, rather than after the system has already accumulated code, tests, and dependencies.[cite:1][cite:4]

SDD also improves maintainability because requirements remain visible in a stable document instead of being buried in conversations or implicit in the codebase.[cite:1][cite:15] That makes

onboarding easier, supports handoffs across roles, and gives reviewers a concrete basis for evaluating whether a change is correct.[cite:4][cite:12][cite:15]

Common benefits include:[cite:1][cite:4][cite:12]

- Less ambiguity in implementation decisions.[cite:4][cite:12]
- Less rework caused by misunderstood requirements.[cite:1][cite:4]
- Better alignment across product, engineering, and QA.[cite:4][cite:12]
- Easier derivation of tests from acceptance criteria.[cite:1][cite:12]
- Better support for parallel work once interfaces and responsibilities are defined.[cite:4]
- Stronger control over AI-generated code through explicit constraints.[cite:4][cite:13][cite:15]

SDD versus iterative prompting

Iterative prompting usually begins with a request such as “build this feature,” followed by repeated correction as the model produces drafts.[cite:4][cite:15] This can work well for small or exploratory tasks, but on larger efforts it often stores requirements in chat context rather than in a durable project artifact.[cite:1][cite:15]

SDD changes the workflow by externalizing the requirements into a specification first.[cite:1][cite:4] Instead of correcting the model ad hoc after each generation, the team revises the spec deliberately and asks the implementation to conform to that updated contract.[cite:13][cite:15]

The practical difference is that iterative prompting optimizes for fast local progress, while SDD optimizes for consistency, traceability, and repeatable delivery.[cite:13][cite:15] Prompting says “produce something close and refine it,” whereas SDD says “produce something that satisfies this defined contract.”[cite:4][cite:15]

Operating model

A useful operating model is to treat the product or project owner as the author of intent and the engineering system, including the LLM, as the executor of that intent.[cite:4][cite:12][cite:13] In that framing, the PM or lead defines the problem, scope, constraints, and acceptance criteria; engineering validates feasibility and architecture; the LLM helps implement; and tests verify compliance with the spec.[cite:4][cite:12]

This does not eliminate iteration.[cite:1][cite:4] It changes where iteration happens: the preferred loop is refine the specification, then regenerate or revise the implementation, instead of accumulating patches through loosely connected prompts.[cite:1][cite:15]

Ideology

The ideology behind SDD is that explicit intent should come before execution.[cite:1][cite:3] It treats software delivery as a contract-driven process in which clarity, validation, and shared understanding are more valuable than improvisation.[cite:4][cite:13]

This is especially relevant in AI-assisted engineering because code generation is increasingly cheap, while verification, maintenance, and correction still carry substantial cost.[cite:2][cite:4][cite:13] SDD addresses that imbalance by making the specification authoritative and using implementation as evidence of compliance.[cite:13][cite:15]

Internal reference bullets

- Write the spec before writing the code.[cite:1][cite:3]
- Make the spec the single source of truth.[cite:1][cite:4][cite:15]
- Define requirements, interfaces, constraints, and acceptance criteria up front.[cite:1][cite:3][cite:12]
- Use the spec to align product, engineering, and QA before implementation begins.[cite:4][cite:12]
- Treat code as an implementation artifact, not as the primary location for discovering requirements.[cite:1][cite:4]
- Use specs to constrain LLM output and reduce guesswork.[cite:4][cite:13][cite:15]
- Revise the spec deliberately instead of endlessly patching prompts.[cite:1][cite:15]
- Derive tests and reviews from the spec so correctness is judged against explicit criteria.[cite:1][cite:12]
- Prefer SDD when correctness, maintainability, coordination, or auditability matters.[cite:4][cite:12][cite:15]
- Think of the workflow as: define intent, validate constraints, implement to spec, verify compliance.[cite:4][cite:12][cite:13]